

Trees 5: Problems on Trees

"You can't be that kid standing at the top of the waterslide, overthinking it. You have to go down the chute."

TINA FEY

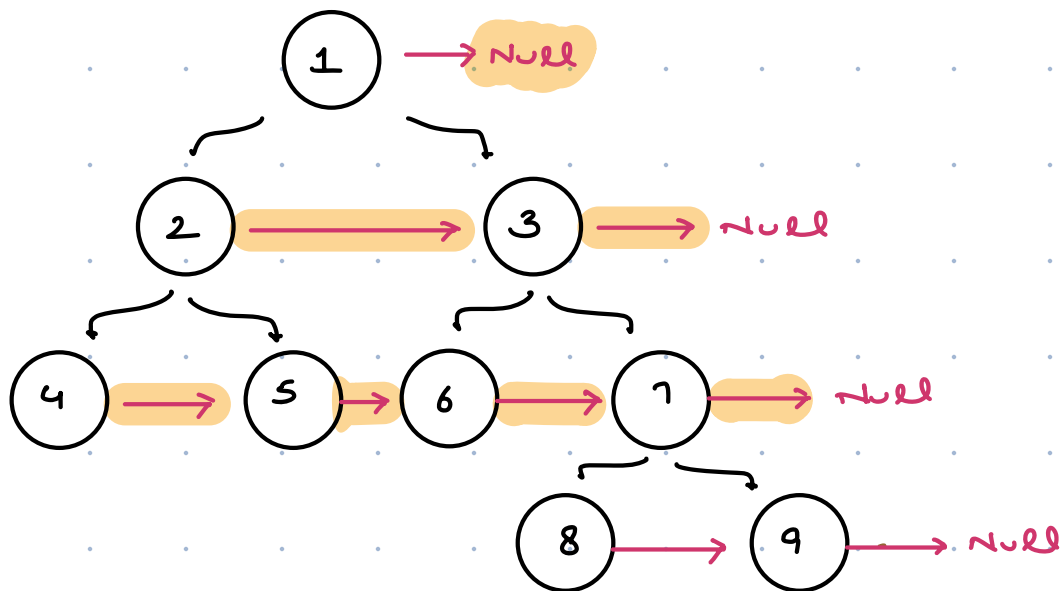
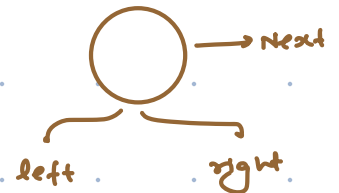


* Content

01. Next pointer in Binary tree
02. Diameter of Binary tree (Amazon)
03. Vertical order Traversal
04. Top view of Binary tree
05. Recover BST

Next pointer in Binary Tree

For all the nodes in tree, next ptr points to null.
Update next ptr to point to the next node in same level. (left to right)



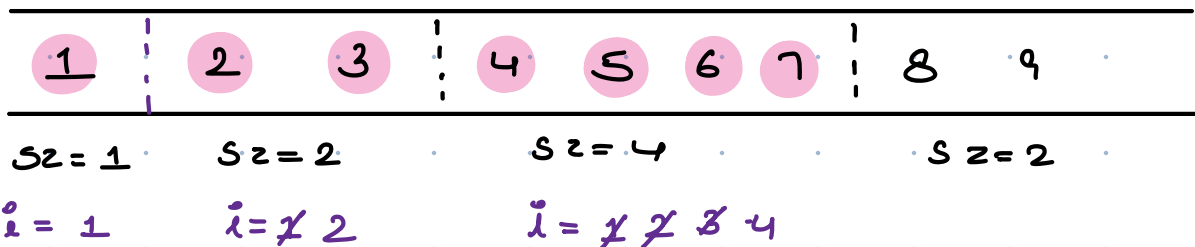
class TreeLinkNode {

int val;

TreeLinkNode left, right, next;

}

queue =>



```

public class Solution {
    public void connect(TreeLinkNode root) {
        Queue<TreeLinkNode> q=new LinkedList<>();

        q.add(root);

        while(q.size()>0){
            int sz=q.size();

            for(int i=1;i<=sz;i++){
                TreeLinkNode rem=q.remove();
                if(i==sz)rem.next=null;
                else{
                    rem.next=q.peek();
                }

                if(rem.left!=null) q.add(rem.left);
                if(rem.right!=null) q.add(rem.right);
            }
        }
    }
}

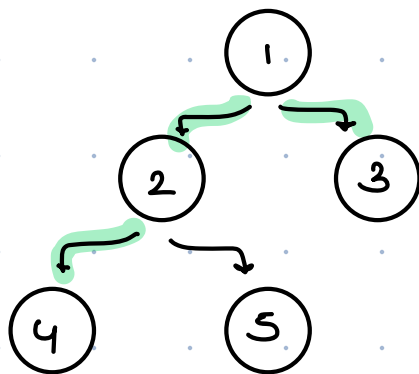
```

TC: $O(N)$

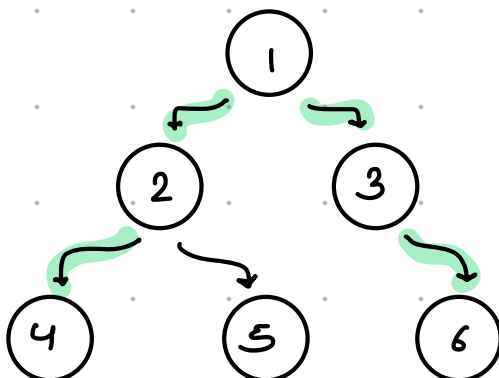
SC: $O(N)$

* Diameter of Binary Tree

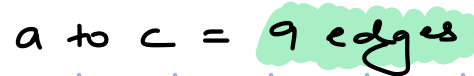
→ No. of the edges along the longest path between any two leaf nodes in the tree.



Ans = 3



Ans = 4



→ Dia can also pass via root Node ✓

```
return Math.max(lh, rh) + 1;
```

```
public int diameter ( root ) {
```

```
    if (root == Null) return 0;
```

$\approx O(n^2)$

```
    int lh = height (root.left);
```

```
    int rh = height (root.right);
```

```
    return Math.max ( diameter (root.left) ,  
                      diameter (root.right) ,  
                      (lh + rh + 2) );
```

* Travel & Change

```
int dia = 0
```

```
int height ( root ) {
```

```
    if (root == Null) return -1
```

```
    int lh = height (root.left)
```

TC : $O(n)$

```
    int rh = height (root.right);
```

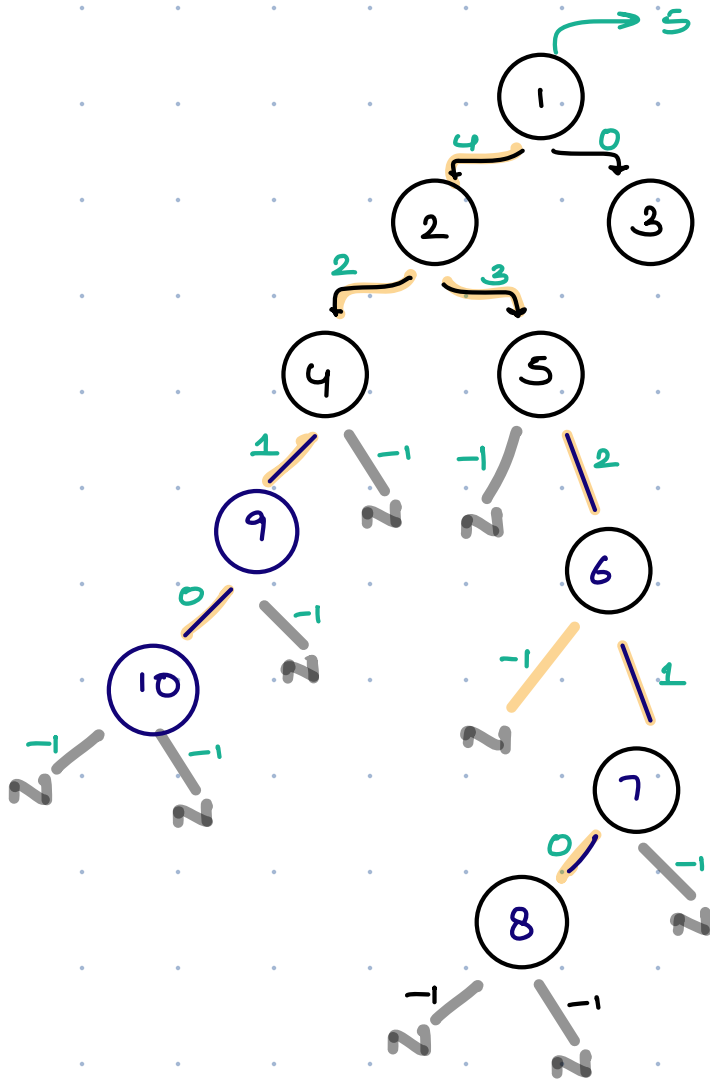
SC : $O(h+1)$

```
    dia = Math.max ( dia, lh + rh + 2 );
```

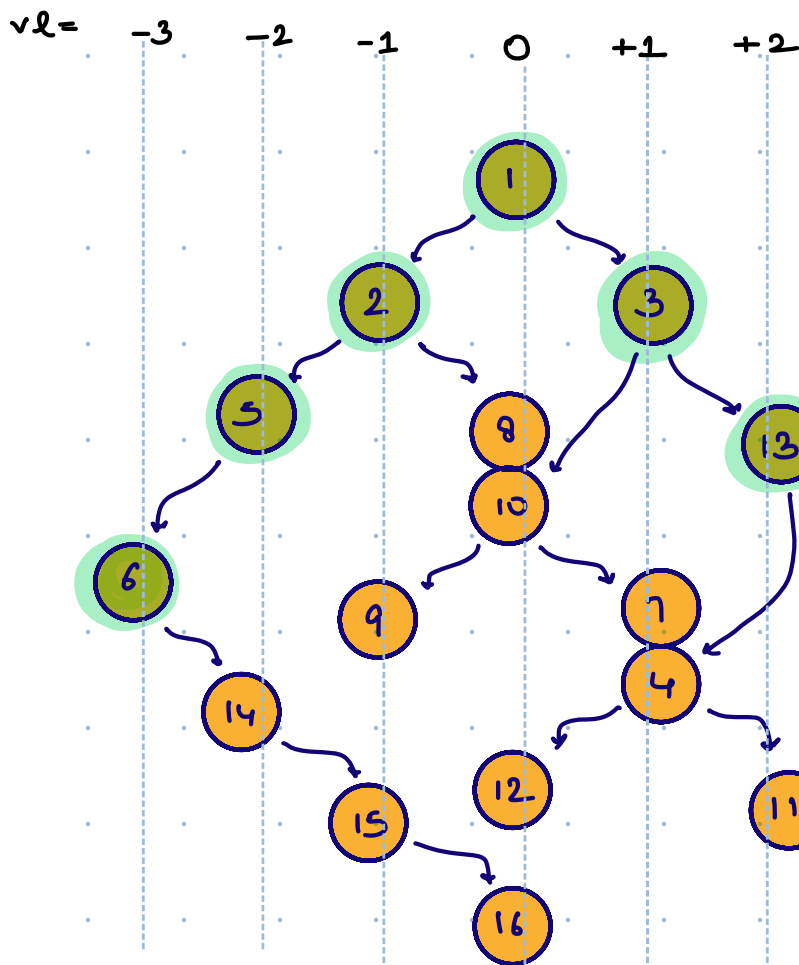
```
    return Math.max ( lh, rh ) + 1;
```

10:10 $\rightarrow$ 10:20 pm

$dia = \emptyset \neq \neq \neq \neq \neq$



Vertical Order Traversal



HM

Int vs AL

-3 ⇒ 6

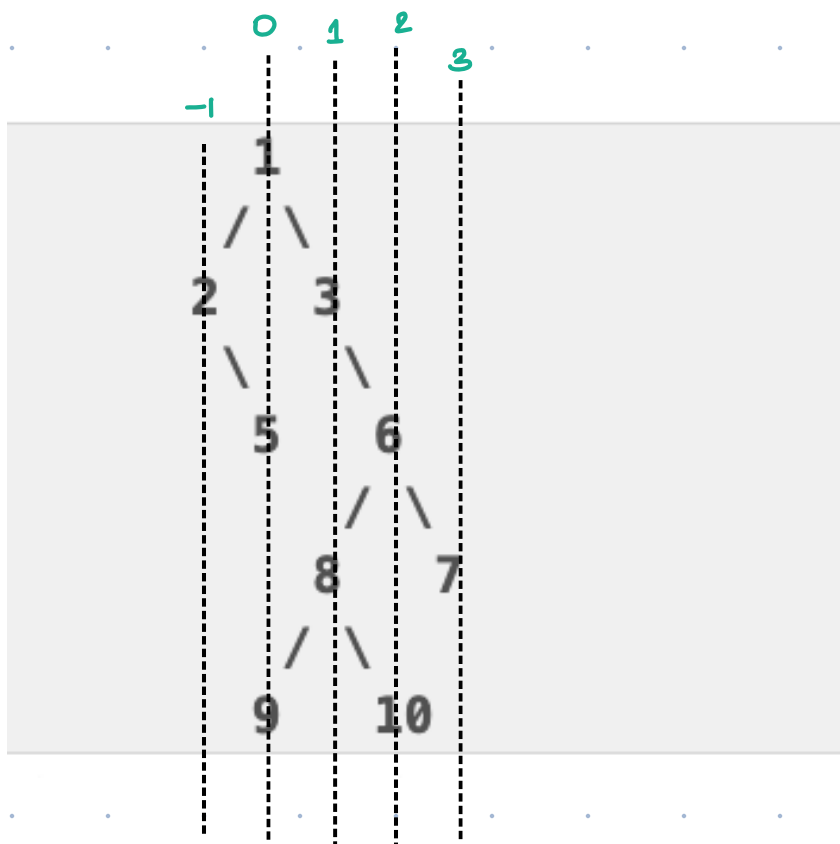
-2 ⇒ 5 14

-1 = 2 9 15

0 = 1 8 10 12 16

1 = 3 7 4

2 = 13 11



-1 = 2

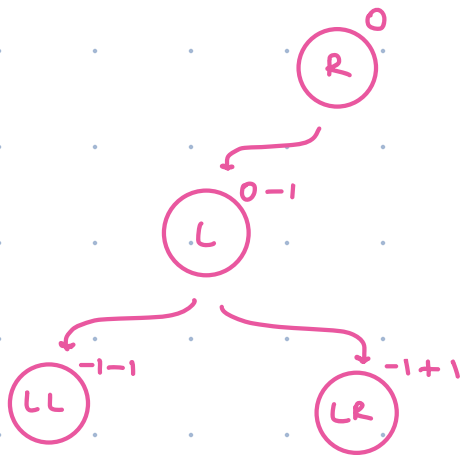
0 = 1 5 9

1 = 3 8

2 = 6 10

3 = 7

01. Vertical levels

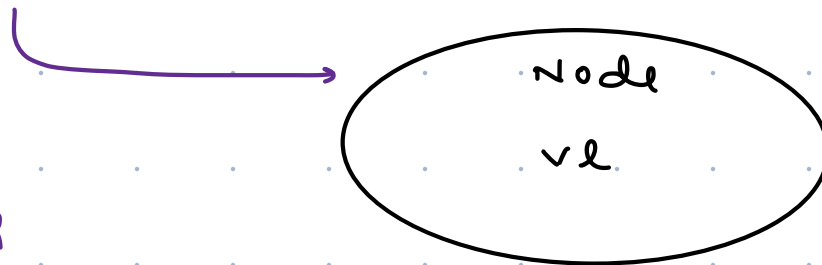


02. HashMap

key = Integer

value = ArrayList<Node>

03. Queue <pair> q



class pair {

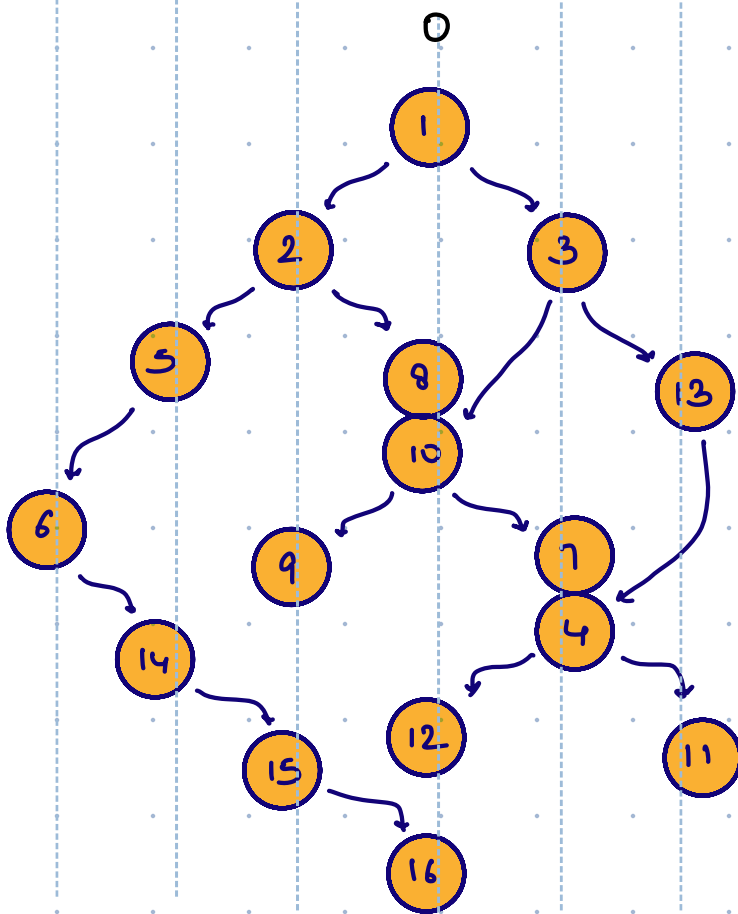
TreeNode node;

int vl;

pair(TreeNode n, int l)

node = n;

vl = l;



HM

$0 \rightarrow 1, 8$
 $-1 \rightarrow 2$
 $1 \rightarrow 3$
 $-2 \rightarrow 5$

~~1, 0~~ ~~2, -1~~ ~~3, 1~~ ~~5, -2~~ ~~8, 0~~

10, 0 13, 2 6, -3

node = 8

vl = 0

```
Queue < pair > q = new LinkedList < >();
```

```
HashMap < I, AL < Node > > hm = new HashMap < >();
```

```
minvl = 0
```

```
maxvl = 0
```

```
pair rp = new pair (root, 0)
```

```
q.add(rp);
```

```
while (q.size() > 0) {
```

```
    pair rem = q.remove();
```

```
    TreeNode remN = rem.node;
```

```
    int reml = rem.vl;
```

```
    minvl = Math.min(minvl, reml);
```

```
    maxvl = Math.max(maxvl, reml);
```

```
    hm.putIfAbsent (reml, new ArrayList < >());
```

```
    hm.get(reml).add(remN);
```

```
    if (remN.left != Null) {
```

```
        pair lc = new pair (remN.left, reml - 1);
```

```
        q.add(lc);
```

```
    if (remN.right != Null) {
```

```
        pair rc = new pair (remN.right, reml + 1);
```

```
        q.add(rc);
```

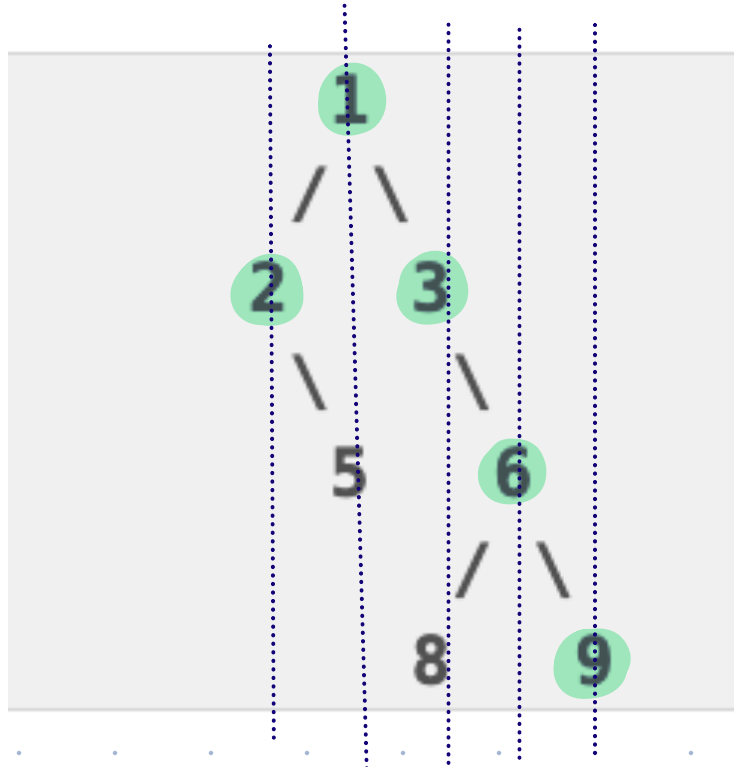
```
    }
```

```

for (int i = minval; i ≤ maxval; i++) {
    AL<N> list = hm.get(i);
    for (Node n : list)
        Print(n.val);
    println();
}

```

* Top view



Top view

2 1 3 6 9

Top view = First Node of AL at every vl

Bottom view = Last Node of AL at every AL

* Recover sorted arr[]

Given an $A[]$, which is formed by swapping 2 distinct index positions in an increasing sorted $arr[]$. Get the original array back.

$$A[] = \{ 4 < 10 < 19 > 14 < 18 > 12 < 25 < 28 \}$$

0 1 2 3 4 5 6 7

$$A[] = \{ 2 < 6 < 10 < 23 > 19 < 20 > 14 \}$$

Original $A[]$

$$A[] = \{ 2, 6, 10, 14, 19, 20, 23 \}$$

$$A[] = \{ 3 < 6 < 9 < 31 > 14 < 18 < 24 > 12 < 35 \}$$

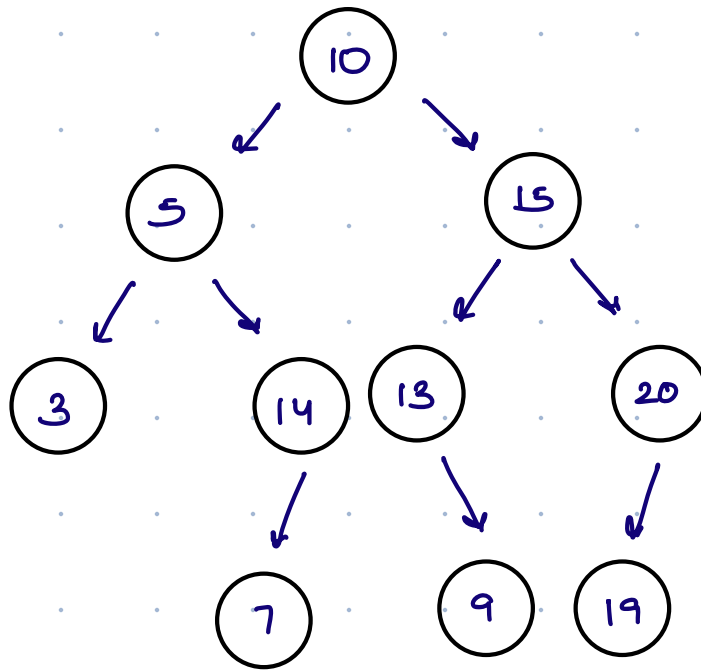
Idea $\rightarrow$ While iterating on arr , we compare two adjacent elements.

$$\text{Ideal comparison} = \text{prev} < \text{curr}$$

01. 1st time comparison fails $\rightarrow$ 1st val = $A[i]$
2nd val = $A[i+1]$

02. 2nd time comparison fails $\rightarrow$ 2nd val = $A[i+1]$

Q3 Given a BST, which is formed by swapping two distinct nodes, recover original BST



Idea 1 = Inorder traversal for BST is sorted.

→ Traverse in inorder, store elements inside an AC. Now, follow the same above approach

TC: $O(n)$

SC: $O(n)$

Idea 2 → Use an extra variable to store previous node & compare it.

static Node prev = null; // help us in comparison with curr element.

static Node first = null, second = null;

void recover (Node root) {

if (root == null) return;

recover (root.left)

if (prev != null && prev.val > root.val && first == Null) {

first = prev

second = root;

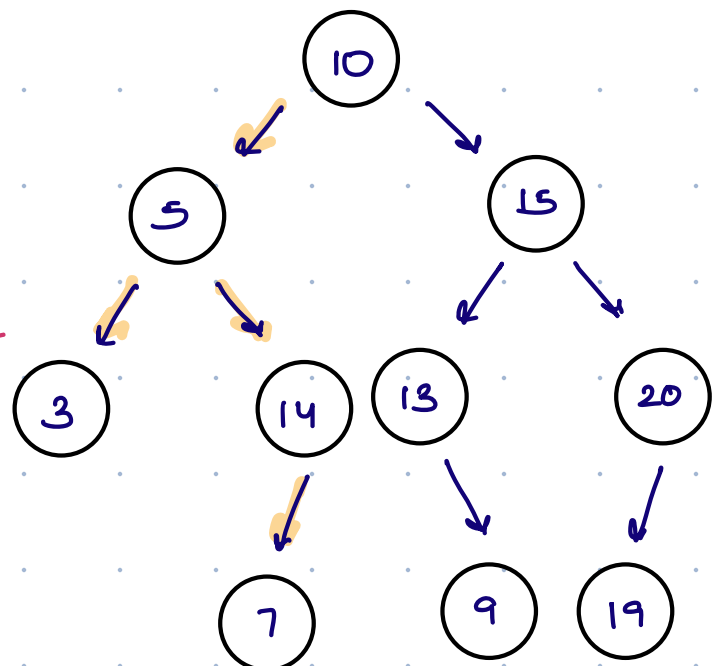
else if (prev != null && prev.val > root.val) {

second = root;

prev = root;

recover (root.right)

R



prev = ~~5~~ 7 14 10 13 9 15
14 20

F = 14

S = ~~10~~ 9